

What is a Loop?

A **loop** is used to repeat a block of code multiple times.

Instead of writing the same code again and again, a loop runs it automatically until a condition is satisfied.

Types of Loops

1. **For Loop** – repeats code a fixed number of times.
 2. **While Loop** – runs while a condition is true.
 3. **Do-While Loop** (C++/Java only) – executes at least once, then checks condition.
-

General Syntax

Python

```
# For loop
for i in range(1, 6):
    print(i)

# While loop
i = 1
while i <= 5:
    print(i)
    i += 1
```

C++ / Java

```
// For loop
for (int i = 1; i <= 5; i++) {
    cout << i;
}

// While loop
int i = 1;
while (i <= 5) {
    cout << i;
    i++;
}

// Do-while loop
int i = 1;
do {
    cout << i;
    i++;
} while (i <= 5);
```

Question 1 — Prime Number Check

A **prime number** is a number greater than 1 that has only two divisors:

- 1
- itself

Example: 2, 3, 5, 7, 11

Approach 1 — Brute Force

Logic

1. If $n \leq 1 \rightarrow$ Not Prime.
2. Check divisibility from 2 to $n-1$.
3. If divisible anywhere $\rightarrow$ Not Prime.
4. Otherwise $\rightarrow$ Prime.

Complexity

Time: $O(n)$

Space: $O(1)$

C++ Code

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    bool isPrime = true;

    if (n <= 1) {
        isPrime = false;
    } else {
        for (int i = 2; i < n; i++) {
            if (n % i == 0) {
                isPrime = false;
                break;
            }
        }
    }

    if (isPrime)
        cout << "Prime";
    else
```

```
        cout << "Not Prime";

    return 0;
}
```

Java Code

```
import java.util.Scanner;

public class PrimeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        boolean isPrime = true;

        if (n <= 1) {
            isPrime = false;
        } else {
            for (int i = 2; i < n; i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }

        if (isPrime)
            System.out.println("Prime");
        else
            System.out.println("Not Prime");

        sc.close();
    }
}
```

Python Code

```
n = int(input())
is_prime = True

if n <= 1:
    is_prime = False
else:
    for i in range(2, n):
        if n % i == 0:
            is_prime = False
            break

if is_prime:
    print("Prime")
else:
    print("Not Prime")
```

Approach 2 — Optimized Method

Instead of checking till $n-1$, check only till $\sqrt{n}$.

Why?

If $n = a \times b$, one factor must be $\leq \sqrt{n}$.

So checking till $\sqrt{n}$ is enough.

Logic

1. If $n \leq 1 \rightarrow$ Not Prime.
 2. Loop from 2 to $\sqrt{n}$.
 3. If divisible $\rightarrow$ Not Prime.
 4. Otherwise $\rightarrow$ Prime.
-

Complexity

Time: $O(\sqrt{n})$

Space: $O(1)$

C++ Optimized Code

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    int n;
    cin >> n;

    bool isPrime = true;

    if (n <= 1) {
        isPrime = false;
    } else {
        for (int i = 2; i <= sqrt(n); i++) {
            if (n % i == 0) {
                isPrime = false;
                break;
            }
        }
    }

    cout << (isPrime ? "Prime" : "Not Prime");
    return 0;
}
```

Java Optimized Code

```
import java.util.Scanner;

public class PrimeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        boolean isPrime = true;

        if (n <= 1) {
            isPrime = false;
        } else {
            for (int i = 2; i <= Math.sqrt(n); i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }

        System.out.println(isPrime ? "Prime" : "Not Prime");
        sc.close();
    }
}
```

Python Optimized Code

```
import math

n = int(input())
is_prime = True

if n <= 1:
    is_prime = False
else:
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            is_prime = False
            break

print("Prime" if is_prime else "Not Prime")
```

Final Summary

Method	Time Complexity
Brute Force	$O(n)$
Optimized	$O(\sqrt{n})$

Switch Statement — Complete Notes (C++, Java, Python)

What is a Switch Statement?

A **switch statement** is used to select one block of code among many options based on the value of a variable.

It is an alternative to writing multiple `if-else if` conditions when checking the same variable.

How it Works

1. A variable/expression is evaluated.
 2. Its value is compared with different cases.
 3. Matching case executes.
 4. `break` stops further execution.
 5. `default` runs if no case matches.
-

General Syntax (C++ / Java)

```
switch(expression) {  
    case value1:  
        // code  
        break;  
  
    case value2:  
        // code  
        break;  
  
    default:  
        // code  
}
```

C++ Example

Program to print day name using number.

```
#include <iostream>  
using namespace std;  
  
int main() {  
    int day;  
    cout << "Enter day number: ";  
    cin >> day;  
  
    switch(day) {  
        case 1:
```

```
        cout << "Monday";
        break;
    case 2:
        cout << "Tuesday";
        break;
    case 3:
        cout << "Wednesday";
        break;
    case 4:
        cout << "Thursday";
        break;
    case 5:
        cout << "Friday";
        break;
    case 6:
        cout << "Saturday";
        break;
    case 7:
        cout << "Sunday";
        break;
    default:
        cout << "Invalid day";
}

return 0;
}
```

Java Example

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int day = sc.nextInt();

        switch(day) {
            case 1:
                System.out.println("Monday");
                break;
            case 2:
                System.out.println("Tuesday");
                break;
            case 3:
                System.out.println("Wednesday");
                break;
            case 4:
                System.out.println("Thursday");
                break;
            case 5:
                System.out.println("Friday");
                break;
            case 6:
                System.out.println("Saturday");
                break;
            case 7:
                System.out.println("Sunday");
                break;
            default:

```

```
        System.out.println("Invalid day");
    }

    sc.close();
}
}
```

Switch in Python

Python traditionally did not have switch, but **Python 3.10+** introduced `match-case`.

Syntax (Python)

```
match variable:
    case value:
        code
```

Python Example

```
day = int(input("Enter day number: "))
```

```
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case 3:
        print("Wednesday")
    case 4:
        print("Thursday")
    case 5:
        print("Friday")
    case 6:
        print("Saturday")
    case 7:
        print("Sunday")
    case _:
        print("Invalid day")
```

`case _` works like default.

Important Points

- `break` prevents fall-through.
- `default` runs if no case matches.
- Switch works with integers, characters, and strings (language dependent).
- Useful when many conditions depend on one variable.